

C-DAWN: Keeping a Battery-Limited UAV Swarm Connected While It Surveys a Himalayan Disaster Valley

Team C-DAWN · [Member names] · [Institution, City]

Source code: github.com/paramvansh01/drone_swarm · Simulation track · Report date: 26 September 2026

ABSTRACT. When an earthquake or a landslide cuts the road and the towers in a Himalayan valley, the first hours depend on two things: getting eyes on the damaged sites, and getting what those eyes see back to the people coordinating the response. We built C-DAWN, a simulation-first autonomy stack for a fleet of battery-limited quadrotors that fly from a ground control station (GCS) set up outside the affected area. The swarm works out from the terrain how many relays it needs, decides which aircraft fly relay and which survey, hands a relay station to a fresh aircraft *before* the tired one leaves to swap its battery, and drops lower-priority work when a new emergency is reported. It is never told about a failure, an outage or a fault. It notices them from heartbeats, acknowledgements, noise readings and battery current, the same evidence a fielded swarm would have. We tested it on four scenarios, two of them on real terrain at Kedarnath and Uttarkashi, each with 4 to 11 hidden disturbances. C-DAWN delivered 97.3% of survey tasks, 96.7% of packets and 95.3% connectivity. A fixed-role baseline managed 87.8%, 91.6% and 88.6%. Neither had a collision, a geofence violation or a flat battery in any run.

Keywords: UAV swarm, BVLOS, multi-hop relay, disaster response, energy-aware planning, fault recovery, delay-tolerant networking, terrain-aware RF

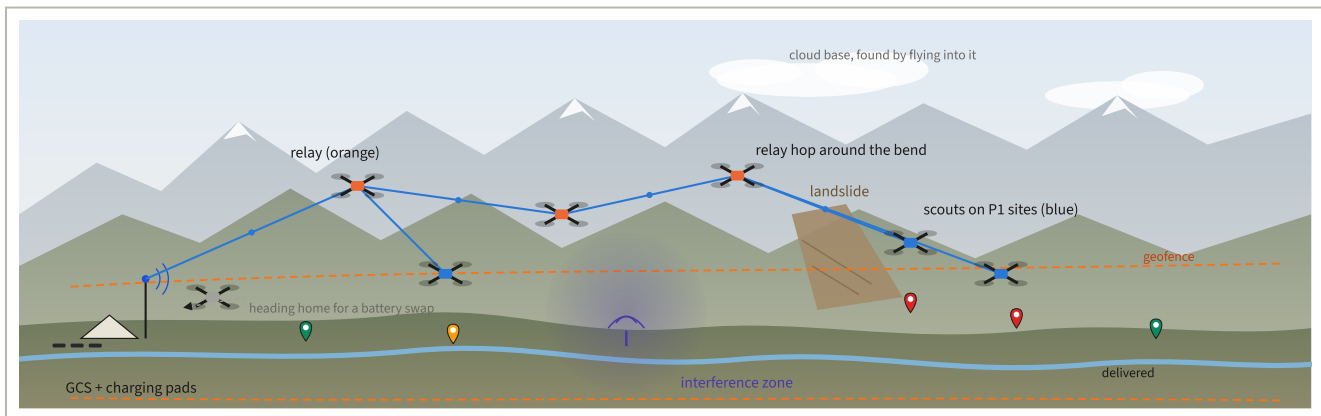


Fig. 1. What C-DAWN is doing, drawn by hand. The GCS sits at the valley mouth, outside the affected area, with its charging pads beside it. Relays (orange) hold a chain that follows the valley round its bend; scouts (blue) survey the reported sites, which are coloured by priority and turn green once their data has reached the GCS. One aircraft is heading back for a battery swap while its station stays covered. The interference zone, the cloud base and the geofence are all things the swarm has to find out about or respect on its own.

1 INTRODUCTION

Mountain valleys are hard places to keep a radio link alive. A 900 MHz link with half a watt of power reaches tens of kilometres in open air, but in the Mandakini or Bhagirathi valleys a single spur of rock can take 40 dB out of it. Add a battery that lasts a quarter of an hour, a valley wind that makes the flight home twice as slow as the flight out, and the ordinary ways a field deployment goes wrong (an aircraft goes down, a radio dies, somebody's generator starts spraying noise across the band), and the problem the UAV-X brief describes becomes clear. The fleet has to survey every reported site and keep talking to the GCS the whole time, and it has to do this without anyone telling it what has just broken.

We treat communication as part of the mission, not a side condition. A site only counts as done when its imagery has reached the GCS, so the swarm is scored on exactly what the responders need. Every decision in the stack, from which aircraft to send to where the relays should hover, is made with the link home in mind.

This report describes what we built and how it performs. Our main contributions are:

1. A **terrain-aware relay requirement**: the swarm walks a chain from the GCS antenna along the valley and places a hop wherever the previous one runs out of margin, so the number of relays comes from the rock and the radio, not from a constant.
2. **Dynamic role management** in which any aircraft can fly any role, with launches from the pads, failover in about 200 to 300 ms, and a **make-before-break handover** so a relay leaving to recharge never opens a gap.

- 3. **Energy- and wind-aware planning** that decides when to come home and which tasks are affordable from the aircraft's own measured power draw.
- 4. A **perception-only awareness layer**: the autonomy never reads the simulator's truth, and a test audits the source code to keep it that way.
- 5. A reproducible evaluation with a JSON scenario format, hidden disturbances, a standard log set and a fixed-role baseline for comparison.

2 PROBLEM AND SYSTEM MODEL

Area and GCS. The affected area is a valley corridor about 4.2 km long inside a 5.1 × 5.1 km terrain tile. The GCS is a fixed ground node at the valley mouth with its antenna on a 10 m mast and a row of landing and charging pads. It cannot fail or run flat, it is where every packet is going, and every aircraft starts and ends its sorties on one of its pads.

Fleet. Five or six identical 1.9 kg quadrotors with a 900 MHz mesh radio (27 dBm, 3 dBi). Hover endurance is about 18 minutes and cruise about 15. A battery swap takes up to two minutes on the pad. Roles are not fixed: an aircraft can be a scout, a relay, or on standby (flying home, or waiting on the pad).

Tasks. Each survey task p has a position, a priority $\pi_p \in \{1, 2, 3\}$ with weight $w_p = 3, 2, 1$, and a release time. Tasks released after launch are the "newly emerging" ones; the swarm cannot see them before they are reported. A task is surveyed after a two-second dwell within 30 m, and delivered when all 12 chunks of its data have arrived at the GCS.

Constraints. A mission clock (12 or 15 minutes in our scenarios), a polygon geofence with an AGL ceiling, at least 3 m between aircraft, and a battery that must never run out in the air. We read "complete the mission safely and within the allotted time" strictly: every aircraft has to be back on a pad by the deadline.

Table 1. How each requirement in the UAV-X brief is met, and how we measure it.

THE SWARM MUST	WHAT C-DAWN DOES	WHERE	MEASURED AS
Survey all assigned locations	Assign a task only if the aircraft can reach it, survey it and get home inside its battery and the clock; greedy on priority weight per second of travel.	sim/guidance.py sim/energy.py	completion rate and time, priority-weighted score
Maintain end-to-end comms with the GCS	GCS as a fixed node; relay chain sized from the terrain; GNN placement; SCM-aware routing; packet-level store-and-forward.	mesh/roles.py mesh/traffic.py	PDR, latency, availability, downtime
Dynamically assign relays	Relay count re-planned every second; launch from pads first, otherwise promote the scout doing the least valuable work.	mesh/roles.py	relay reallocations, re-configuration efficiency
Reconfigure on degradation, failure or recharge	Heartbeat-based failover; interference localised and routed round; make-before-break relay handover; lost-link failsafe.	mesh/election.py mesh/awareness.py	recovery time, performance after failures
Prioritise new high-priority regions	A new P1 task pre-empts the best-placed scout on lower-priority work; standby scouts; discretionary returns turned round.	sim/guidance.py	emergency response time, weighted score
Finish safely, on time	Energy-aware return home; home before the deadline; geofence projection on every setpoint; predictive separation; sequenced take-offs.	sim/deconfliction.py sim/world.py	collisions, minimum separation, flat batteries, geofence breaches

3 ARCHITECTURE

Everything runs in one Python process on one simulated clock. The live dashboard, the three-laptop demonstration and the headless benchmark all call the same

`step()`, so the numbers we report come from exactly the code a judge would watch. Fig. 2 shows the layers. The part we care most about is the dashed line in the middle. Below it is the simulator, which knows everything. Above it is the autonomy, which only knows what its sensors and its radio tell it.

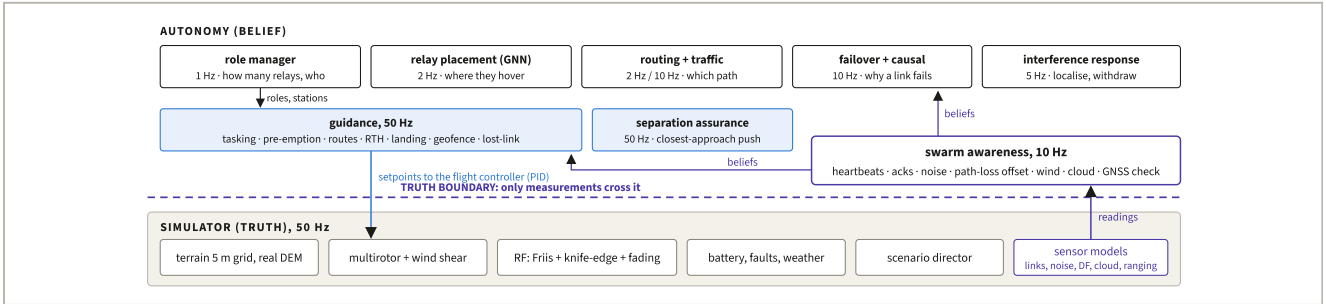


Fig. 2. Layers, rates and the truth boundary. The simulator (bottom) owns terrain, physics, the radio channel, faults and the scenario's hidden disturbances. Only sensor readings cross the purple line. The awareness layer turns them into beliefs, and every autonomy layer above decides from those beliefs. Setpoints are the only thing that goes back down. Rates are per simulated second; physics runs at 50 Hz.

4 SIMULATION ENVIRONMENT

Terrain. Each site is a 5.1 km terrain pack with a 5 m elevation grid built from AWS Open Data terrain tiles (SRTM and Copernicus derived) and Sentinel-2 cloudless imagery [9, 10]. We ship four real disaster sites, Kedarnath (2013 debris flow), Uttarkashi (the 1991 earthquake area), Joshimath and Chungthang, plus a procedural valley for repeatable tests. The mission corridor is found by dynamic programming along the valley floor.

Flight. An underactuated multirotor model with attitude lag, slew limits and airspeed drag, flown by a cascaded PID. Wind follows a power-law shear on height above ground (exponent 0.16) with Dryden turbulence and gusts [8].

Radio. Every tick, every pair of aircraft and every aircraft-to-GCS link gets a link budget:

$$P_r = P_t + G_t + G_r - L_{fs}(d) - J(v) - L_x, \quad SNR = P_r - N \quad (1)$$

where L_{fs} is Friis free-space loss [2], L_x is extra loss (wet antennas in rain) and $J(v)$ is the ITU-R P.526 single knife-edge diffraction loss over the worst ridge on the path [1]. Here h is how far the ridge pokes above the straight radio path, and d_1, d_2 are its distances to the two ends:

$$J(v) = 6.9 + 20 \log_{10}(\sqrt{((v - 0.1)^2 + 1)} + v - 0.1) \quad (2)$$

$$v = h \sqrt{(2(d_1 + d_2) / (\lambda d_1 d_2))}$$

Packet delivery ratio is averaged over a burst of 24 packets, each with its own Rician fade ($K = 6$ dB). We learnt early that a single fading draw per tick makes a perfectly steady link look like it is flapping, which confused both routing and the causal engine.

5 MISSION AUTONOMY

Before the individual pieces, it helps to see one aircraft's life (Fig. 3). It starts charged on a pad, launches as a scout or a relay, may swap roles in the air, and eventually comes home for one of five reasons. It lands, has its battery swapped, and waits until the role manager needs it again. Everything below decides when one of these arrows is taken.

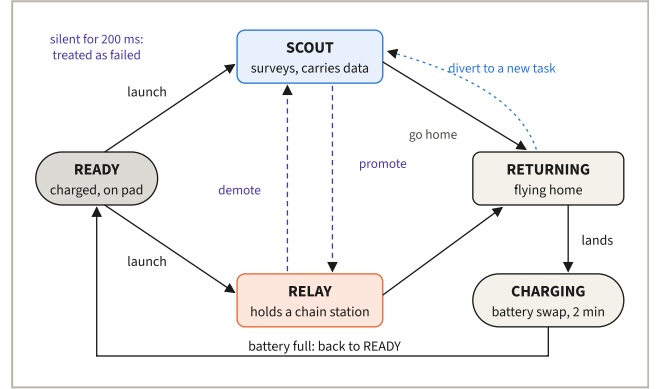


Fig. 3. The life of one aircraft. Solid arrows are autonomous transitions; dashed purple arrows are role changes in flight (each one a logged reallocation). An aircraft goes home for one of five reasons: its energy threshold, the mission clock, a lost link, no work left, or a completed handover. A failure is not a state the swarm is told about. An aircraft nobody has heard for 200 ms is simply treated as gone.

5.1 Energy and time, before anything else

Every decision that sends an aircraft away from the GCS has to be affordable, so the energy model answers three questions: how much battery it takes to get home from here, whether a task can be done and still leave enough to get home, and when a relay has to call for its replacement. Drain follows the airframe's own law, scaled by a factor $\hat{k}$ that each aircraft measures from its battery current:

$$\dot{b} = \hat{k} [b_0 + (b_1 - b_0)f_T], \quad v_g = \text{clip}(v_a + \hat{\mathbf{w}} \cdot \hat{\mathbf{u}}, 5, 19) \quad (3)$$

with f_T the thrust fraction, $\hat{\mathbf{w}}$ the estimated wind at cruise height and $\hat{\mathbf{u}}$ the direction of travel. Leg time is $T(a, b) = 1.15|ab|/v_g + 25$ s. The return-home and handover thresholds then follow directly, with a safety factor $s = 1.2$ and an 8% landing reserve:

$$B_{rth} = \max(12, s \dot{b} T(\mathbf{p}, \mathbf{h}) + 8) \quad (4)$$

$$B_{ho} = B_{rth} + s \dot{b}_{hov} (T(\mathbf{g}, \mathbf{q}) + 25 \text{ s}) \quad (5)$$

Here $\mathbf{p}$ is where the aircraft is, $\mathbf{h}$ its home pad, $\mathbf{g}$ the GCS and $\mathbf{q}$ a relay's station. The point of Eqs. (4) and (5) is that there is no fixed "come home at 22%" rule. A relay four kilometres up the valley with the wind in its face turns for home at about 62%, one next to the GCS at 12%. Fig. 4 shows the curves.

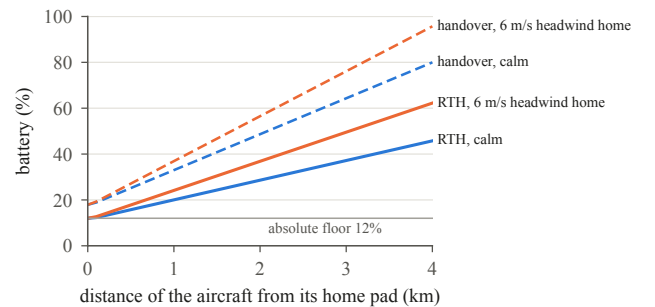


Fig. 4. Thresholds from Eqs. (4) and (5) for a cruising aircraft. A headwind on the way home adds 16 points of battery at 4 km. Dashed lines are the relay handover thresholds, which leave time for a replacement to fly out.

5.2 Who surveys what, and when to change our mind

Allocation is greedy on value rate. At each step we take the (scout, task) pair with the largest

$$w_p / (ETA_{s,p} + 60 s) \quad (6)$$

subject to: task affordable, and round trip $\leq$ time left

The 60 s term stops a very close P3 task from always beating a P1 task a little further away. When a P1 task arrives and no idle scout can take it, the best-placed scout on lower-priority work is pre-empted, unless it is already hovering over its target. Idle scouts with a healthy battery loiter at a standby point mid-valley, inside relay cover, so the next emergency is minutes closer. A scout that is flying home only because it had nothing to do can be turned round. Fig. 5 is the whole loop.

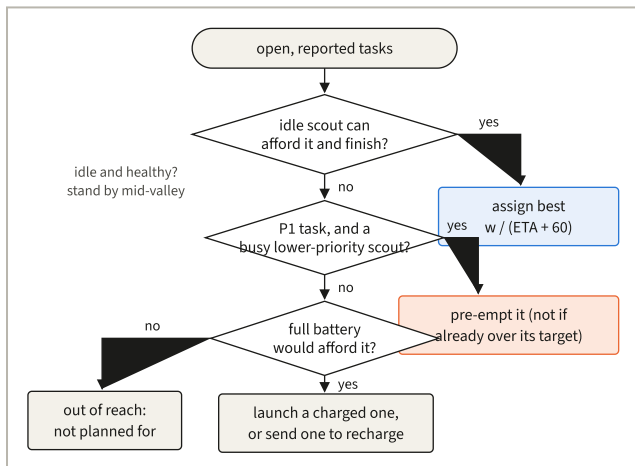


Fig. 5. The allocation loop, run every tick. Affordability means battery for the task plus the trip home plus the landing reserve; finishing means the whole round trip fits before the deadline.

5.3 How many relays the terrain asks for

This is the piece we are proudest of, because it replaces a guess with a measurement of the mountain. Once a second the role manager collects the points where work is happening: scouts on task, open reachable tasks, aircraft carrying undelivered data, and aircraft still flying home. For the farthest of them it walks the chain in Algorithm 1.

Algorithm 1: Terrain-aware relay chain

1. Sample the valley centreline every 60 m from the GCS to the target, at relay height (160 m AGL).
2. **while** the current hop cannot reach the target with SNR ≥ 15 dB:
3. pick the *farthest* sample the current hop still reaches with SNR ≥ 15 dB, using Eq. (1) over the terrain map, the measured noise floor and any interference source the swarm has localised;
4. make it a hop; **if** no sample is reachable, stop.
5. **return** the hop count (the relay requirement) and the hop points (anchors for the GNN).

The 15 dB target is roughly the 9 dB a 256-byte frame needs for 95% delivery, plus fading margin. On the Kedarnath scenario the walk gives exactly one relay

(Fig. 6). The valley turns south past the 2 km mark and a spur blocks the straight line from the GCS to the far sites.

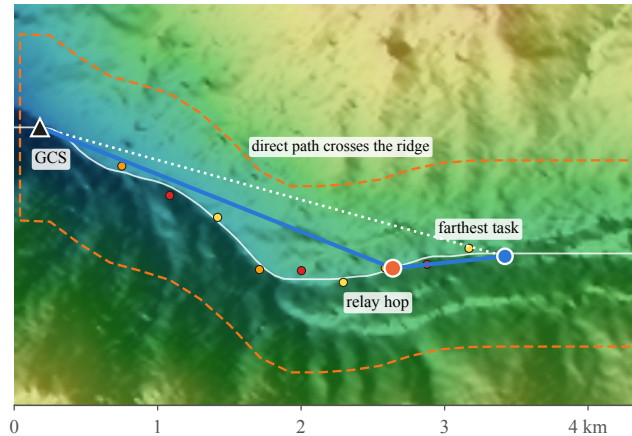


Fig. 6. Algorithm 1 on the real Kedarnath terrain. Blue lines are the planned links; the white dotted line is the direct path, which clips the spur. Dots are survey tasks by priority (red P1, amber P2, yellow P3); the orange dashed line is the geofence.

The requirement then drives assignment. When there are fewer relays than needed, a charged aircraft on a pad is launched first, because it arrives with a full battery. Failing that, the scout doing the least valuable work that is closest to the missing hop is promoted, and at least one scout always keeps surveying. When there are more relays than needed for 25 s, the spare goes back to surveying, or home if there is nothing to survey. Every change is logged as a reallocation with its reason; one that is undone within 30 s counts as a flap against reconfiguration efficiency.

5.4 Where the relays hover

The role manager decides how many relays and who flies them; the placement network decides where. An E(3)-equivariant graph network [3] proposes stations in one forward pass, then 20 Adam steps on a differentiable version of Eq. (1), with terrain sampled by `grid_sample`, polish them. The hop points from Algorithm 1 are the starting anchors. On 48 held-out layouts over six unseen terrains, connectivity went from 0.48 (naive) to 0.58 with the network alone and 0.66 after refinement, in about 1.4 ms plus 21 ms. We should be honest that the refinement does most of the work. The network's real job is giving it a good place to start.

5.5 Handing over a relay station

Batteries are the reason relays leave, and a naive swarm pulls the relay home and then notices the hole. We do it the other way round (Fig. 7). When a relay's battery reaches B_{ho} from Eq. (5), the role manager launches a charged aircraft from the pads, or promotes a scout, and sends it to the relay's exact station. The outgoing relay is taken out of the placement plan and holds where it is. It leaves only once the replacement is holding station. If something goes wrong and its own return threshold arrives first, it comes home anyway. Safety beats coverage.

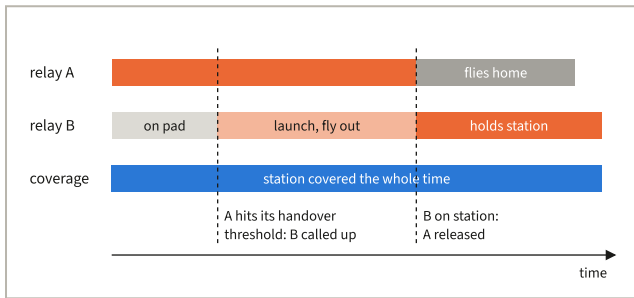


Fig. 7. Make-before-break. The overlap between the two dashed lines is the price of never opening a gap; Eq. (5) sizes it so that A can afford to wait and still get home.

5.6 Failover, routing and data that has to wait

Failover. Each aircraft's radio is "heard" when anyone, another aircraft or the GCS, measures a usable link to it. A relay nobody has heard for 200 ms is treated as failed and the best-scoring scout (battery times mean link quality) is promoted into its slot. The router recomputes at once. Detection dominates the latency, as Fig. 8 shows: in our logged runs the whole thing, from the relay going silent to traffic flowing on the new route, took 200 to 300 ms. Nothing that still works is ever demoted. The old election re-ranked the whole swarm on every failure, and we saw a healthy relay get pushed out of its role because of it.

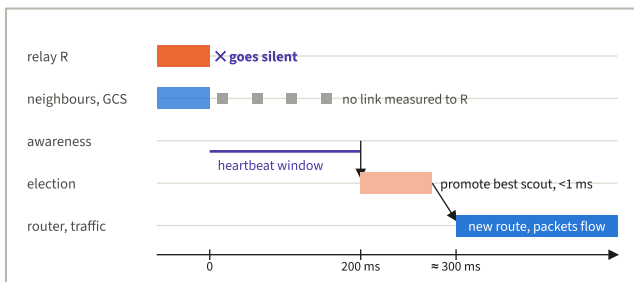


Fig. 8. Failover, as it happens. Nobody announces that relay R has failed. The swarm notices that nobody can hear it, and after 200 ms the election fills the slot and the router moves traffic. The election itself takes microseconds; the waiting is what costs time.

Routing. Dijkstra on $-\log(\text{PDR})$, blended with a stability estimate from the causal layer, with the GCS as a node in the graph [4].

Traffic. Packets actually move. Every airborne aircraft sends telemetry at 2 Hz (dropped once stale, after 2 s), and each surveyed site produces 12 chunks of imagery and situational data. Each hop is tried up to four times, and every try costs airtime, so a bad link shows up as latency before it shows up as loss. Survey chunks are delay-tolerant [7]: with no route they wait in custody on whichever aircraft holds them, and an aircraft that lands hands its backlog to the GCS over the wire. Every delivered packet is acknowledged, and those acknowledgements are the only way an aircraft knows it still has a link home. Sixty seconds without one triggers the lost-link return. Fig. 9 shows the three ways survey data can travel.

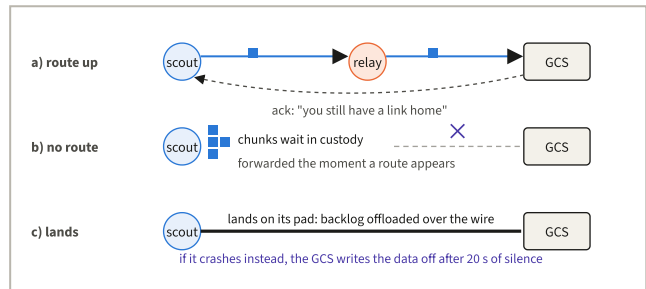


Fig. 9. How survey data gets home. Telemetry uses path (a) only and is dropped after 2 s. Survey chunks use whichever of (a), (b) or (c) is available, and a task counts as delivered only when all twelve of its chunks have arrived.

5.7 Safety

Every setpoint is projected at least 40 m inside the geofence polygon and under the AGL ceiling before the controller sees it. Separation assurance predicts each pair's closest point of approach over the next four seconds. If the predicted miss is under 30 m (8 m near the pads), it pushes the pair apart in proportion to the intrusion and splits them vertically, then re-applies the fence and the terrain floor. Take-offs from the pads are spaced four seconds apart. Across all 24 benchmark runs there were no collisions, no geofence violations, no terrain strikes and no aircraft that ran flat.

6 KNOWING WITHOUT BEING TOLD

A simulator makes cheating easy. The first version of this code, like a lot of swarm demos, read the injected fault directly: "if this drone is marked dead, re-elect". That makes a fault look handled when a real swarm would still be finding out about it. We went back and removed every such path. A disturbance now changes the physics or the radio and nothing else. No flag is set and no callback fires. Table 2 lists what the swarm actually sees.

Table 2. How each hidden disturbance is noticed.

DISTURBANCE	EVIDENCE THE SWARM HAS	WHAT IT CONCLUDES
UAV failure	No one hears it for 200 ms	Failover; task released after 5 s; its data written off after 20 s
Radio outage	Same silence; its own packets go unacknowledged	Treated as a failure; the aircraft comes home after 60 s and rejoins when heard
Interference zone	Receiver noise readings; direction-finding bearings	Source cross-fixed; the <i>estimate</i> enters relay planning
Area-wide RF drop, packet loss, link cut	Noise floor rises everywhere; link measurements fall	Chain re-sized; routes and relays re-planned
Battery or motor fault	Current draw above what the thrust should need	Drain factor $\hat{k}$ rises; earlier return, fewer tasks accepted
Heavy rain	RSSI below the map's prediction; camera loses the ground	Extra path loss planned for; ceiling set by flying into cloud
GNSS drift	Radio ranging to other aircraft and to the GCS disagrees with GNSS	Terrain-relative navigation after 2 s
New emergency	The report reaches the GCS	Tasking, pre-emption, chain extended

One consequence surprised us at first and then seemed obviously right. A relay whose radio fails and a relay that crashes look identical from the outside, so the swarm handles both the same way. When the radio comes back, the aircraft is heard again, rejoins, and the role manager notices it has one relay too many. A test file audits the autonomy modules' source code for any read of simulator state, and 20 behavioural tests check that each disturbance is noticed rather than announced.

Honesty cost us a little. When we switched the energy model from the true wind to the swarm's own estimate, completion dropped from 98.4% to 93.5%. The cause was a calibration fitted against ground-level wind, while the swarm now measures the stronger wind at cruise height. Refitting airspeed against the same logged flights brought it back to 97.3%.

7 EVALUATION

7.1 Scenarios and baseline

A scenario is a JSON file that fixes the terrain, seed, fleet, GCS, geofence, tasks, clock and a timeline of hidden disturbances. Positions can be given in metres or as a fraction along the valley, so one scenario works on any site. The disturbance types cover everything the Stage 2 brief lists. Table 3 summarises the four we ship.

Table 3. Shipped scenarios (tasks: at launch + released later).

SCENARIO	UAVS	TASKS	CLOCK	HIDDEN EVENTS
Quickstart, synthetic valley	5	8 + 1	12 min	4
Kedarnath landslide	6	10 + 6	15 min	8
Uttarkashi earthquake	5	9 + 6	15 min	7
Stress, synthetic valley	5	8 + 7	15 min	11

The **baseline** keeps everything except the adaptive role layer: same flight control, energy-aware return, geofence, relay placement network and packet model. Roles are fixed at launch, one relay and the rest scouts, and aircraft relaunch in their original role. A failed relay is not replaced, a new P1 task waits for a free scout, nothing hands over and no one stands by. The gap between the two is what the adaptive autonomy is worth. Each scenario runs with three seeds, which vary radio fading, packet outcomes and, in the stress scenario, task placement.

Metrics follow the brief's categories. The priority-weighted score is

$$S = \sum_p w_p \delta_p / \sum_p w_p \quad (7)$$

where $\delta_p = 1$ if task p 's data reached the GCS by the deadline. The rest are defined in Table 4. They are computed from the simulation itself, and never from what the swarm believes.

Table 4. Metrics, grouped as in the brief.

GROUP	METRIC	DEFINITION
Mission	completion rate	tasks whose data reached the GCS before the deadline, over tasks released
	emergency response	release of a new task to its data arriving at the GCS
Comms	packet delivery ratio	delivered over generated, telemetry and survey data together
	latency	creation to arrival at the GCS, per telemetry packet
	availability, downtime	airborne time with (and without) an end-to-end path of reliability ≥ 0.5
Autonomy	reallocations	every role change, logged with its reason
	recovery time	disruption until every airborne aircraft has had a path for 1 s; zero if none lost it
Robustness	reconfiguration eff.	1 minus the share of role changes undone within 30 s
	after faults	availability and PDR from the first disruption on
Safety	collisions, separation	pairs closer than 3 m (per incident); closest approach
	charge, geofence	aircraft that ran flat; excursions outside the polygon or ceiling

7.2 Results

Fig. 10 compares the two strategies over all 12 runs each. The adaptive swarm is ahead on every mission and communication metric. The biggest gap is how quickly a new emergency gets answered: 42 s against

180 s, because the baseline has to wait for a scout to become free while ours pre-empts one or turns one round. Downtime and recovery time are both roughly halved, because the chain is re-sized and re-staffed instead of being left as it was launched.

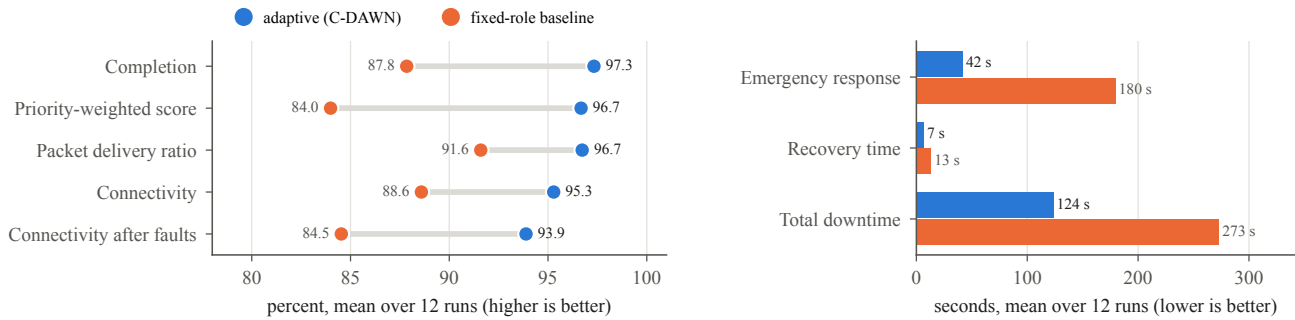


Fig. 10. Adaptive C-DAWN against the fixed-role baseline, mean of 12 runs each (4 scenarios × 3 seeds). Left: percentages (dots, since the axis is zoomed). Right: times in seconds. Neither strategy had a collision, a geofence violation or a flat battery.

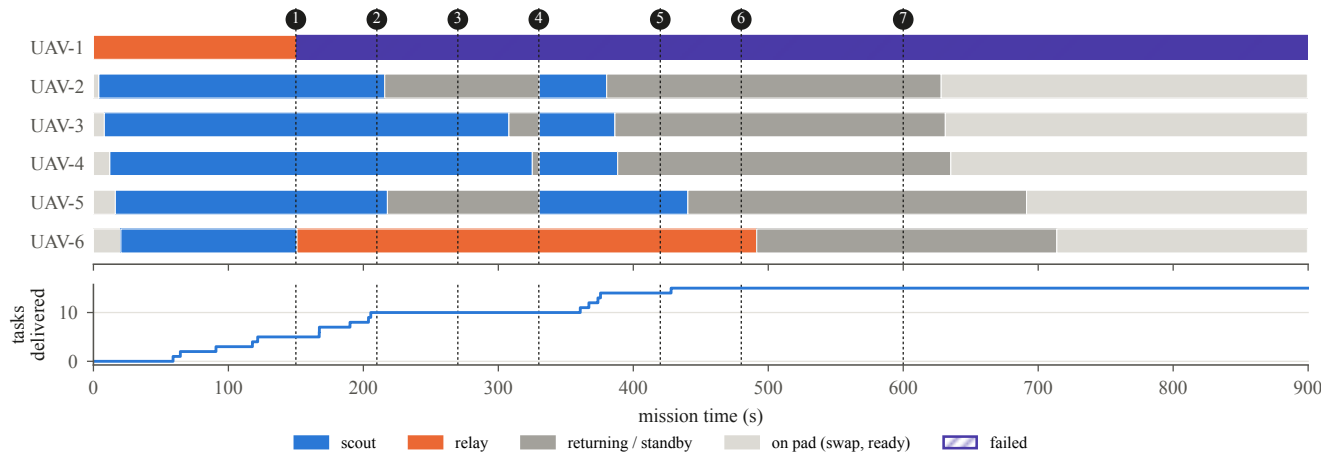


Fig. 11. One Kedarnath run, taken from its logs. Disturbances: 1 relay UAV-1 fails, 2 GCS receiver outage, 3 30% packet loss, 4 P1 region reported, 5 interference zone, 6 relay battery fault, 7 late P1 report. At 1 the swarm notices UAV-1’s silence and promotes UAV-6 into its slot. At 4 two scouts that were heading home are turned round, and the region is delivered by about 430 s. After 6 the faulty relay’s measured drain rises and it comes home early. Event 7 arrives with 300 s left, and its round trip does not fit, so the planner declines it.

Table 5. Adaptive results per scenario (mean of 3 seeds).

SCENARIO	COMPL.	PDR	AVAIL.	RECOV.	MIN BATT.
Quickstart	100.0%	95.4%	93.9%	4.6 s	24.1%
Kedarnath	93.8%	97.2%	95.8%	8.9 s	10.5%
Uttarkashi	100.0%	100.0%	100.0%	0.0 s	29.4%
Stress	95.6%	94.3%	91.5%	13.3 s	21.3%

Table 5 breaks the adaptive numbers down, and Fig. 11 follows one Kedarnath run from start to finish. Three observations stand out. Uttarkashi is a draw with the baseline. The Bhagirathi valley there is straight and open, one relay covers everything, and adaptation has little to win, which is what it should look like. The stress scenario is the hardest on communication, and its 13 s mean recovery is dominated by a 60 s severed link and a 25 s GCS outage that no amount of re-planning can fix, only wait out. Finally, the adaptive swarm flies its batteries harder, down to 10.5% in Kedarnath

against 34% for the baseline on average. That is still above the 8% landing reserve, and it is the price of keeping aircraft working instead of parking them.

7.3 What did not work

We think a report is more useful when it says what went wrong. Our first energy model was about 1.8 times too pessimistic and sent full-battery scouts home before they had done anything. Tasks were once left assigned to a scout that had died, so nobody else picked them up. The liquid time-constant controller [5] we originally proposed to fly the aircraft lost to a well-tuned cascaded PID in 16 of 16 paired gust trials (0.62 m against 0.18 m cross-track error), so the PID flies and the LTC runs in shadow. The causal link diagnosis [6] never blames terrain for interference, but it confirms genuine terrain shadow only about 20% of the time. The geometry is against it: climbing one end of a link lifts the path over a far ridge by very little.

8 SOFTWARE AND REPRODUCIBILITY

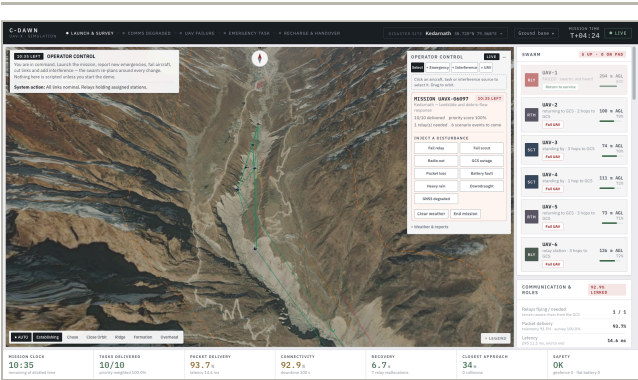


Fig. 12. The operator console during a live Kedarnath run: real terrain, the geofence, the relay chain, survey sites by priority, the swarm list with hop counts, the communication and roles panel, and the challenge metrics along the bottom. Disturbances can be injected live from the same console.

The code is Python 3.10+ with PyTorch for the placement network, FastAPI and WebSockets for the GCS server, and React with Three.js for the console. Nothing needs a GPU or an internet connection once installed: terrain packs and trained weights ship in the repository. Three commands cover what a judge needs.

```
# watch a mission live in the console
python run_node.py --autostart \
  --scenario kedarnath_landslide
# one headless run, 5 to 9x real time
python run_scenario.py \
  scenarios/uttarkashi_earthquake.json
# all scenarios, 3 seeds, both strategies
python -m bench.uavx_suite
```

For a live demonstration the same code can be split across three laptops on one Wi-Fi network (Fig. 13). This is not three copies of one screen: work really moves between machines, and it moves back if a machine goes quiet.

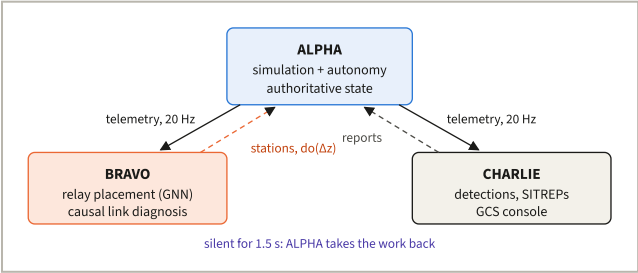


Fig. 13. Three-laptop deployment. ALPHA owns the aircraft. BRAVO and CHARLIE compute on the live stream and push results back, which ALPHA applies. Every laptop serves the full console, so any of them can go on the projector.

Every run writes a standard log set: run metadata with the code version, the resolved scenario, an event stream, 1 Hz aircraft state and link tables, a per-packet log and the metrics. When the organisers publish their log format, one module, `mission/recorder.py`, is the

adapter point. There are 158 automated tests, covering the physics, the radio model, the placement network's equivariance, failover, handover, traffic, every disturbance type and the truth-boundary audit.

9 LIMITATIONS AND STAGE 2 PLAN

This is a simulation. C-DAWN runs on its own Python simulator, which the brief allows as an equivalent to PX4, Gazebo or ns-3, because it lets terrain, radio and flight share one model. The autonomy only ever talks to an aircraft through setpoints, so for Stage 2 we plan a PX4 SITL and ROS 2 bridge that replaces the physics and controller hooks and leaves the autonomy untouched. Three other things are on the list: a contention-aware MAC model, since today airtime is modelled per hop but not shared; the organisers' log format; and position estimation from a proper filter rather than the true position plus an injected GNSS offset. Air density does not change with altitude yet. Survey detections come from each site's category rather than from imagery, and the situation reports are templates.

10 CONCLUSION

A swarm that surveys a disaster valley is only useful if what it sees gets home. By treating the link to the GCS as part of the mission, sizing the relay chain from the terrain, handing relay stations over before a battery forces the issue, and making the swarm find out about trouble the way a fielded one would, C-DAWN delivered 97% of its tasks through 30 hidden disturbances without losing an aircraft to its own planning. The next step is putting the same autonomy on PX4 and, after that, on real airframes.

REFERENCES

1. ITU-R, "Propagation by diffraction," Recommendation P.526-15, 2019.
2. H. T. Friis, "A note on a simple transmission formula," *Proc. IRE*, vol. 34, no. 5, pp. 254 to 256, 1946.
3. V. G. Satorras, E. Hoogeboom and M. Welling, "E(n) equivariant graph neural networks," in *Proc. ICML*, 2021.
4. İ. Bekmezci, O. K. Sahingoz and Ş. Temel, "Flying ad-hoc networks (FANETs): A survey," *Ad Hoc Networks*, vol. 11, no. 3, pp. 1254 to 1270, 2013.
5. R. Hasani, M. Lechner, A. Amini, D. Rus and R. Grosu, "Liquid time-constant networks," in *Proc. AAAI*, 2021.
6. J. Pearl, *Causality: Models, Reasoning and Inference*, 2nd ed. Cambridge University Press, 2009.
7. K. Fall, "A delay-tolerant network architecture for challenged internets," in *Proc. ACM SIGCOMM*, 2003.
8. U.S. Department of Defense, "Flying qualities of piloted aircraft," MIL-F-8785C (Dryden turbulence model), 1980.
9. European Space Agency, "Copernicus DEM," and AWS Open Data Terrain Tiles, accessed 2026.
10. EOX IT Services, "Sentinel-2 cloudless," s2maps.eu, CC BY 4.0.
11. J. Scherer *et al.*, "An autonomous multi-UAV system for search and rescue," in *Proc. ACM DroNet*, 2015.